

Assignment 1:-

Create a React component called PlaylistCard that accepts song name and artist as props and displays them, then render three PlaylistCard components with different data to practice passing props.

Answer: ;-

PlayList.jsx

```
import React from "react";

const PlaylistCard = ({ songName, artist }) => {
  return (
    <div style={{ border: "1px solid gray", padding: "10px", margin: "10px" }}>
      <h2>{songName}</h2>
      <p>Artist: {artist}</p>
    </div>
  );
};

export default PlaylistCard;
```

App.jsx

```
import React from "react";
import PlaylistCard from "../PlaylistCard";

const App = () => {
  return (
    <div>
      <h1>My Playlist</h1>

      <PlaylistCard
        songName="Shape of You"
        artist="Ed Sheeran"
      />

      <PlaylistCard
        songName="Believer"
        artist="Imagine Dragons"
      />

      <PlaylistCard
        songName="Blinding Lights"
        artist="The Weeknd"
      />
    </div>
  );
};
```

```
    );  
  };  
  
  export default App;
```

2. Build a simple LikeCounter component that uses useState to track and display the number of likes, and increments the count each time a 'Like' button is clicked, similar to Instagram.

Answer:-

LikeCounter.jsx

```
import React, { useState } from "react";  
  
const LikeCounter = () => {  
  const [likes, setLikes] = useState(0);  
  
  const handleLike = () => {  
    setLikes(likes + 1);  
  };  
  
  return (  
    <div>  
      <h2>💖 Likes: {likes}</h2>  
  
      <button onClick={handleLike}>  
        Like  
      </button>  
    </div>  
  );  
};  
  
export default LikeCounter;
```

App.jsx

```
import React from "react";  
import LikeCounter from "../LikeCounter";  
  
const App = () => {  
  return (  
    <div>  
      <LikeCounter />  
    </div>  
  );  
};  
  
export default App;
```

Question 3. Refactor a given React component tree where a user object is passed down through three levels (App > Feed > Post > LikeButton) via props — update it to use React Context to avoid prop drilling.

Hint: Use createContext and useContext to provide and consume the user object.

Answer:-

UserContext.jsx

```
import { createContext } from "react";

export const UserContext = createContext();
```

App.jsx

```
import React from "react";
import Feed from "../Feed";
import { UserContext } from "../UserContext";

const App = () => {
  const user = {
    name: "Nena Ram",
    role: "React Developer",
  };

  return (
    <UserContext.Provider value={user}>
      <Feed />
    </UserContext.Provider>
  );
};

export default App;
```

Feed.jsx

```
import React from "react";
import Post from "../Post";

const Feed = () => {
  return <Post />;
};

export default Feed;
```

Post.jsx

```
import React from "react";
import LikeButton from "../LikeButton";
```

```
const Post = () => {
  return <LikeButton />;
};

export default Post;
```

LikeButton.jsx

```
import React, { useContext } from "react";
import { UserContext } from "../UserContext";

const LikeButton = () => {
  const user = useContext(UserContext);

  return (
    <button>
      👍 Liked by {user.name}
    </button>
  );
};

export default LikeButton;
```

Question 4. Write a custom hook called useFetchData that accepts a URL and returns loading, data, and error states. Use this hook in a FlipkartProductList component to fetch and display product names from <https://fakestoreapi.com/products>.

Hint: Move repetitive fetch logic out of the component into the custom hook.

Answer:-

useFetchData.jsx

```
import { useEffect, useState } from "react";

const useFetchData = (url) => {
  const [data, setData] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");

  useEffect(() => {
    const fetchData = async () => {
      try {
        setLoading(true);
```

```

    const response = await fetch(url);

    if (!response.ok) {
      throw new Error("Failed to fetch data");
    }

    const result = await response.json();

    setData(result);
  } catch (err) {
    setError(err.message);
  } finally {
    setLoading(false);
  }
};

fetchData();
}, [url]));

return { data, loading, error };
};

export default useFetchData;

```

flipkartProductList.jsx

```

import React from "react";
import useFetchData from "../useFetchData";

const FlipkartProductList = () => {
  const {
    data,
    loading,
    error,
  } = useFetchData(
    "https://fakestoreapi.com/products"
  );

  if (loading) {
    return <h2>Loading...</h2>;
  }

  if (error) {
    return <h2>Error: {error}</h2>;
  }
}

```

```
return (  
  <div>  
    <h1>Product List</h1>  
  
    <ul>  
      {data.map((product) => (  
        <li key={product.id}>  
          {product.title}  
        </li>  
      ))}  
    </ul>  
  </div>  
);  
};  
  
export default FlipkartProductList;
```

App.jsx

```
import React from "react";  
import FlipkartProductList from "../FlipkartProductList";  
  
const App = () => {  
  return (  
    <div>  
      <FlipkartProductList />  
    </div>  
  );  
};  
  
export default App;
```